

Compatibility Checker

Copilot PoC

Architecture & Design Document

Ask · Search · Know — Before Every Patch

1. Overview

Before patching a Linux system, engineers must manually verify that the target kernel version is certified for every installed enterprise application. This involves searching vendor HCL pages, reading release notes, and cross-checking advisories — a process that takes 30–60 minutes per patch window and is prone to human error.

This PoC eliminates that manual step. An engineer asks a plain-English question in VS Code Copilot Chat and receives a structured answer — verdict, reason, references — in seconds. The system uses per-application source files to guide Copilot to the right vendor documents, and is designed to plug in an LLM API in the future with zero structural changes.

Before (Manual)	After (This PoC)
Open browser, navigate to vendor HCL page	Ask one question in Copilot Chat
Search release notes for kernel version	Get answer in < 30 seconds
Cross-check with Red Hat advisories	Receive references to validate
Document findings in a spreadsheet	Answer is structured and repeatable
30–60 min per patch window	< 1 min per patch window

2. How It Works

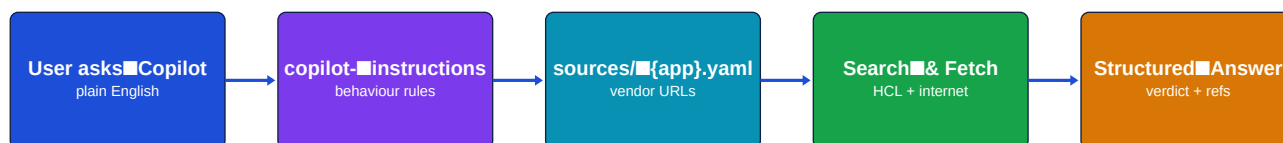


Figure 1 — End-to-end flow

Step-by-step

- 1 Ask** — Engineer types a plain-English question into VS Code Copilot Chat: "Is Veritas InfoScale 8.0.2 compatible with RHEL 8 kernel 4.18.0-477.el8?"
- 2 Instructions** — Copilot reads `.github/copilot-instructions.md`, which tells it exactly how to answer compatibility questions — format, confidence levels, citation style, and when to say UNKNOWN.
- 3 Sources** — Copilot reads `sources/veritas-infoscale.yaml`, which lists the official vendor HCL URL, release notes URL, and recommended search terms for that app.
- 4 Search** — Copilot fetches those URLs and searches the internet using the configured search terms, looking for the certified kernel version range.
- 5 Answer** — Copilot returns a structured answer: COMPATIBLE / NOT COMPATIBLE / UNKNOWN, the reason, specific version bounds, reference URLs, and a patching note.

3. Answer Format

Every answer follows the same structure regardless of the verdict:

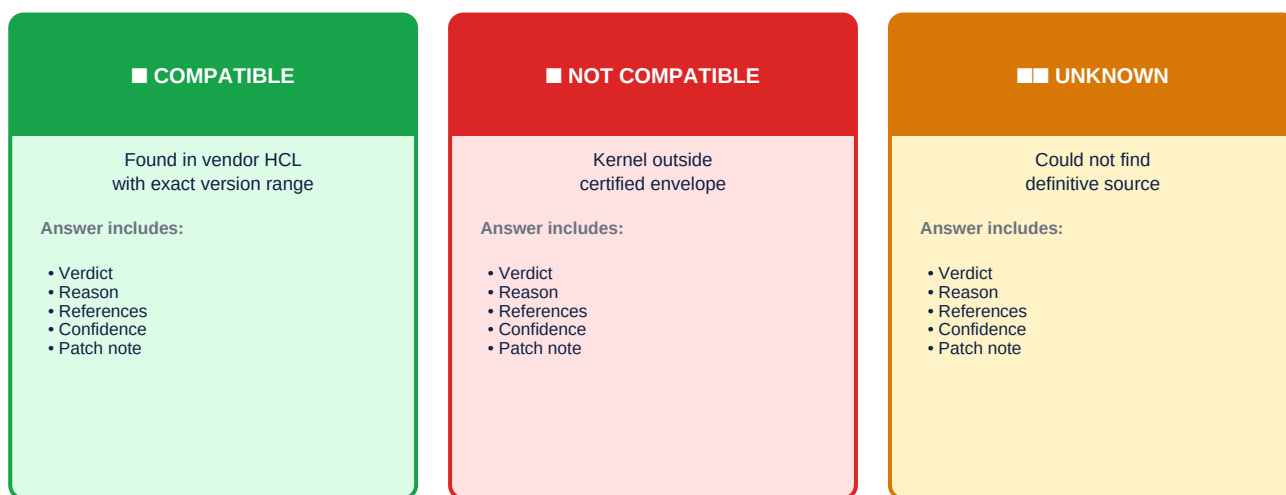


Figure 2 — Three possible verdicts and what each answer contains

Example answer

```
## Compatibility Check Application: Veritas InfoScale 8.0.2 Platform: RHEL 8 Kernel:
4.18.0-477.el8 ### Verdict: NOT COMPATIBLE Reason: Supported range is 4.18.0-193.el8 to
4.18.0-425.el8. Kernel 4.18.0-477.el8 exceeds the upper bound. References: 1. Veritas HCL Oct
2023 — https://veritas.com/... — Section: Supported Kernels RHEL 8 2. Release Notes 8.0.2 —
https://veritas.com/... — Known Limitations Confidence: High Note: Safe maximum is
4.18.0-425.el8. Do not patch beyond this.
```

4. Sources Structure

Each application has its own YAML file in `sources/`. These files tell Copilot exactly where to look — vendor HCL pages, release notes, support portals — and what search terms to use. Adding a new application takes less than 5 minutes.

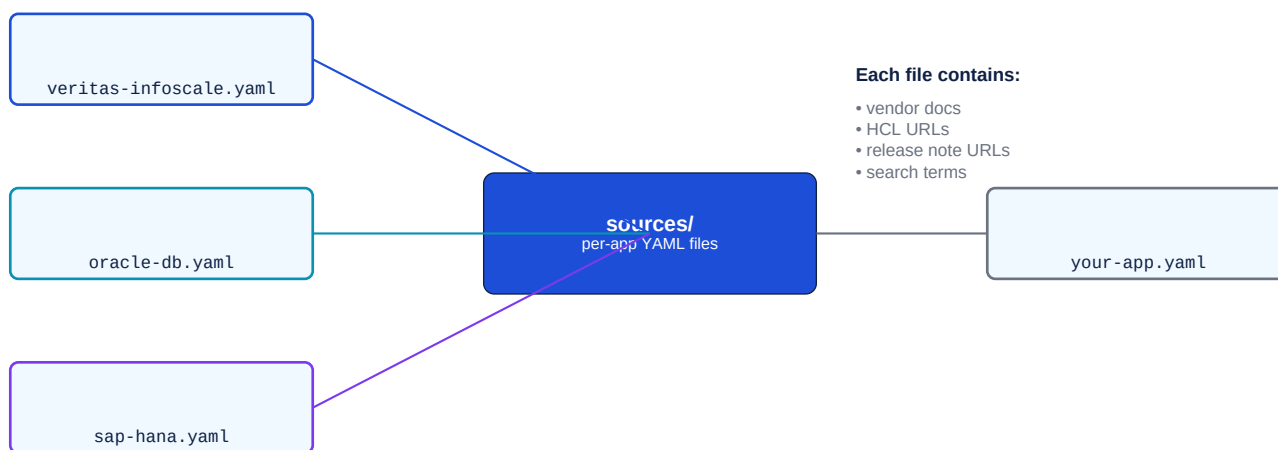


Figure 3 — Per-application source files

Source file structure

```
app: veritas-infoscale vendor: Veritas Technologies versions: [8.0.2, 9.1] platforms: [rhel,
ubuntu, sles] docs: - name: Veritas InfoScale HCL url:
https://www.veritas.com/support/en_US/article/000127013 type: hcl notes: Primary source -
lists supported OS, kernel, hardware - name: Release Notes url:
https://www.veritas.com/support/en_US/article/000127005 type: release_notes search_terms: -
"veritas infoscale {version} rhel {platform_version} kernel compatibility" - "site:veritas.com
infoscale {version} supported kernels"
```

5. Future: LLM API Integration

The PoC is designed so that switching from Copilot to an LLM API requires no structural changes. The sources files, copilot-instructions, and answer format all stay identical. Only the backend changes.

Phase	What changes	What stays the same
Now (PoC)	User pastes question into Copilot Chat manually	sources/ files, answer format, copilot-instructions
Phase 2 (API)	Script calls LLM API with same prompt + sources	sources/ files, answer format, question syntax
Phase 3 (UI)	Web or CLI interface for non-technical users	sources/ files, answer format, LLM backend

When the API key is available, one script wraps it all:

```
# future/query.py – same sources, same prompt, automated
import os, yaml from openai import
OpenAI # or anthropic, azure client = OpenAI(api_key=os.environ['LLM_API_KEY'])
sources =
yaml.safe_load(open('sources/veritas-infoscale.yaml'))
prompt = build_prompt(app, version,
kernel, sources)
answer = client.chat.completions.create(model='gpt-4o', ...)
print(answer) #
same format as Copilot gives today
```